The Performance of 15 Sorting Algorithms in C++

Presentation Review Report

Presenter: Adrian-Andrei Muntoiu

Reviewer: Maria Maiorescu

Faculty of Computer Science
West University of Timișoara

1 Main Results Presented

The presentation investigated the performance of fifteen sorting algorithms implemented in C++. The algorithms included quadratic, comparison-based, non-comparison, and hybrid sorting methods. Experiments were conducted on input sizes ranging from 10 to 100 million elements using different data distributions.

The results showed that non-comparison sorting algorithms, especially Radix Sort and Counting Sort, achieved the best performance on very large datasets. Quick Sort outperformed Merge Sort for large inputs due to better cache locality, while Heap Sort performed worse than expected despite its theoretical complexity. The presentation also demonstrated that practical performance depends not only on asymptotic complexity but also on hardware-related factors such as cache behavior.

2 What Worked Well in the Presentation

- The topic was ambitious and covered a large number of sorting algorithms.
- Experimental results were clearly summarized in tables and comparisons.
- The presenter successfully connected theoretical complexity with practical performance.
- The conclusions were supported by extensive benchmarking data.
- The discussion of cache effects provided useful insight beyond standard algorithm analysis.

3 What Did Not Work Well in the Presentation

- Because many algorithms were included, some of them could not be discussed in great detail.
- Several tables contained a large amount of numerical data, which required careful attention from the audience.
- More visual graphs could have made the performance comparisons easier to follow.

4 What I Learned from the Presentation

I learned that theoretical complexity alone does not always predict real-world performance. Factors such as cache locality and memory access patterns can have a significant impact. I also learned that Radix Sort can be dramatically faster than comparison-based sorting algorithms when sorting large collections of integers.

5 Questions Asked and Answers Received

Question: Why didn't you run the program more times to make sure that the data is accurate?

Answer: The presenter explained that he also wanted to include the $O(n^2)$ algorithms in the benchmark. Since these algorithms already required a considerable amount of time to execute, performing multiple runs for every algorithm and input size would have significantly increased the total execution time. For consistency, he preferred to keep the same testing approach for all algorithms rather than repeating only part of the experiments.